

Author: Gabriel Vance

Class: DSC630 - Predictive Analytics

Professor: Professor Andrew Hua

Date: November 14, 2025

Purpose: This notebook performs the tasks outlined in 10.2 Assignment: Recommender Systems.

Movie Recommender System Implementation

Overview

This project implements a **collaborative filtering** movie recommender system using the MovieLens dataset. The system uses **item-based collaborative filtering** with **cosine similarity** to recommend movies to users based on their preferences.

Methodology

The recommender system works by:

1. **Data Loading & Preprocessing:** Loading movie and rating data, creating user-item matrices
2. **Similarity Calculation:** Computing cosine similarity between movies based on user ratings
3. **Recommendation Generation:** Finding movies most similar to a user's input movie
4. **Filtering & Ranking:** Presenting the top 10 most similar movies

Data Sources

- **Movies Dataset:** Contains movie information (ID, title, genres)
- **Ratings Dataset:** Contains user ratings (userId, movieId, rating, timestamp)

In [1]:

```
# Import required libraries
import pandas as pd
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.feature_extraction.text import TfidfVectorizer
import warnings
warnings.filterwarnings('ignore')

# Suppress warnings for cleaner output
import os

# Suppress DeprecationWarning and TqdmWarning specifically
warnings.filterwarnings("ignore", category=DeprecationWarning)
os.environ["TQDM_DISABLE"] = "1"

from IPython.core.display import display, HTML

display(HTML("""
<style>
/* Make the main container full width */
.container {
    width: 100% !important;
}

/* Wrap code inside code cells */
div.output pre,
div.output code,
div.output_area pre {
    white-space: pre-wrap !important;
    word-wrap: break-word !important;
}

/* Wrap code typed in code cells */
.jp-CodeCell .jp-Editor {
    white-space: pre-wrap !important;
    word-wrap: break-word !important;
}

/* Wrap markdown text and inline code */
.rendered_html pre,
.rendered_html code {
    white-space: pre-wrap !important;
    word-wrap: break-word !important;
}

/* Wrap long outputs (DataFrames, text, errors) */
.output_subarea {
    white-space: pre-wrap !important;
    word-wrap: break-word !important;
}

```

```

/* Optional: prevent horizontal scrolling */
div.output_area {
    overflow-x: auto;
}
</style>
""")

```

Step 1: Load the MovieLens Dataset

The MovieLens dataset consists of four CSV files:

- **Links** — Contains external identifiers for each film, including `imdbId` and `tmdbId`, with `movieId` as the primary key.
- **Movies** — Provides core movie metadata such as `movieId`, `title`, and `genres`.
- **Ratings** — Includes user-generated ratings, with `userId`, `movieId`, `rating` (0.5–5.0), and a `timestamp`.
- **Tags** — Captures user-supplied descriptive tags (e.g., "comedy," "weird") along with `userId`, `movieId`, and `timestamp`.

In [2]:

```

# Load movies data
# Note: Because of the volume of data, we first used a sample set and then u
movies = pd.read_csv('full_data/movies.csv')

print(f"Movies dataset shape: {movies.shape}")
print(f"Sample movies:")
print(movies.head())

#! This dataset is 213 MB, so be mindful of your resources
ratings = pd.read_csv('full_data/ratings.csv')

# Now we can print out some basic information about the ratings dataset
print(f"\nRatings dataset shape: {ratings.shape}")
print(f"Sample ratings:")
print(ratings.head())

# Because there are different users and movies, we'll print the total unique
print(f"\nUnique users: {ratings['userId'].nunique()}")
print(f"Unique movies: {ratings['movieId'].nunique()}")
print(f"Rating range: {ratings['rating'].min()} - {ratings['rating'].max()}")

```

Movies dataset shape: (87585, 3)

Sample movies:

	movieId	title \
0	1	Toy Story (1995)
1	2	Jumanji (1995)
2	3	Grumpier Old Men (1995)
3	4	Waiting to Exhale (1995)
4	5	Father of the Bride Part II (1995)

	genres
0	Adventure Animation Children Comedy Fantasy
1	Adventure Children Fantasy
2	Comedy Romance
3	Comedy Drama Romance
4	Comedy

Ratings dataset shape: (32000204, 4)

Sample ratings:

	userId	movieId	rating	timestamp
0	1	17	4.0	944249077
1	1	25	1.0	944250228
2	1	29	2.0	943230976
3	1	30	5.0	944249077
4	1	32	5.0	943228858

Unique users: 200948

Unique movies: 84432

Rating range: 0.5 – 5.0

Step 2: Data preprocessing and exploration

By merging the **ratings** and **movies** datasets using their shared `movieId`, we create a combined table that links each user rating to the corresponding movie title and genre, enabling more meaningful analysis. We then calculate how many ratings each movie has received and apply a minimum threshold of **10 ratings** to remove films with too little data, since low-rating counts can lead to unstable or unreliable similarity scores. After filtering to include only these sufficiently rated “popular” movies, we generate a quick summary of the most frequently rated titles to confirm the dataset’s structure and gain insight into which films dominate user activity.

In [3]:

```
# Merge movies and ratings data so that we have movie titles along with ratings
movie_ratings = pd.merge(ratings, movies, on='movieId', how='inner')
print(f"Combined dataset shape: {movie_ratings.shape}")

# Check for movies with sufficient ratings (at least 10 ratings for better statistics)
movie_rating_counts = movie_ratings.groupby('movieId').size()
popular_movies = movie_rating_counts[movie_rating_counts >= 10].index
print(f"Movies with at least 10 ratings: {len(popular_movies)}")

# Filter to only include popular movies
filtered_ratings = movie_ratings[movie_ratings['movieId'].isin(popular_movies)]
```

```
print(f"Filtered dataset shape: {filtered_ratings.shape}")

# Display most rated movies
top_movies = filtered_ratings.groupby('title').size().sort_values(ascending=
print(f"\nTop 10 most rated movies:")
print(top_movies)
```

Combined dataset shape: (32000204, 6)
 Movies with at least 10 ratings: 31961
 Filtered dataset shape: (31842705, 6)

Top 10 most rated movies:

title	
Shawshank Redemption, The (1994)	102929
Forrest Gump (1994)	100296
Pulp Fiction (1994)	98409
Matrix, The (1999)	93808
Silence of the Lambs, The (1991)	90330
Star Wars: Episode IV – A New Hope (1977)	85010
Fight Club (1999)	77332
Jurassic Park (1993)	75233
Schindler's List (1993)	73849
Lord of the Rings: The Fellowship of the Ring, The (2001)	73122

dtype: int64

Step 3: Create the User–Item Matrix

To prepare the data for similarity-based recommendations, we convert the filtered ratings into a **user–item matrix**, where each row represents a user, each column represents a movie, and each cell contains that user’s rating for that movie. Ratings that do not exist are filled with `0`, creating a consistent numerical structure required for matrix operations. After generating this matrix, we measure its **sparsity**, which indicates how many of the possible user–movie pairs have no rating—an important characteristic of real-world recommendation data. We then transpose this table to create a **movie–user matrix**, which is used for item-based collaborative filtering, where movies are compared to other movies based on shared user ratings. Finally, we preview a small section of the user–item matrix to verify that the structure looks correct.

In [4]:

```
#!/ Without limiting the dataset, the user-item matrix would be too large to
# I had several kernel crashes before I decided to cap the datasize.
USERS = 3000
MOVIES = 3000

top_users = filtered_ratings['userId'].value_counts().head(USERS).index
top_movies = filtered_ratings['movieId'].unique()

filtered_small = filtered_ratings[
    filtered_ratings['userId'].isin(top_users) &
    filtered_ratings['movieId'].isin(top_movies)]
```

```
]

movie_user_matrix = filtered_small.pivot_table(
    index='userId',
    columns='movieId',
    values='rating',
    fill_value=0
)
print(f"User-item matrix shape: {movie_user_matrix.shape}")
```

User-item matrix shape: (3000, 31945)

Step 4: Calculate Movie Similarity Matrix using Cosine Similarity

This step creates a movie-to-movie similarity matrix using **cosine similarity**, which measures how closely two movies are related based on user rating patterns. Each movie is represented as a vector of user ratings in the `movie_user_matrix`, and cosine similarity compares the angle between these vectors. After computing the similarity scores for all movie pairs, the results are stored in a DataFrame (`movie_similarity_df`) for easy lookup. Finally, the code prints the matrix shape and displays the top similarity scores for the first movie in the dataset.

In [5]:

```
# Calculate cosine similarity between movies
# We use the movie-user matrix where each row represents a movie and columns
movie_similarity = cosine_similarity(movie_user_matrix.T)

# Convert to DataFrame for easier handling
movie_similarity_df = pd.DataFrame(
    movie_similarity,
    index = movie_user_matrix.columns,
    columns = movie_user_matrix.columns
)

print(f"Movie similarity matrix shape: {movie_similarity_df.shape}")

# Display sample similarities for first movie
sample_movie_id = movie_similarity_df.index[0]
print(f"\nSimilarity scores for movie ID {sample_movie_id}:")
print(movie_similarity_df.loc[sample_movie_id].sort_values(ascending=False).)
```

Movie similarity matrix shape: (31945, 31945)

Similarity scores for movie ID 1:

movieId

1 1.000000

3114 0.922731

1270 0.920119

480 0.917703

1198 0.912760

260 0.910607

Name: 1, dtype: float64

This cell can be ran multiple times to see examples of movies (note, you don't have to include the year in the search)

In [6]:

```
# Show 10 random movies to pick from
movies.sample(10)[['movieId', 'title', 'genres']]
# Note: This is purely for getting random movies to explore similarities with
```

Out [6]:

	movieId	title	genres
77394	257707	South Korea: Earth's Hidden Wilderness (2018)	Documentary
28876	132420	Babar: King of the Elephants (1999)	Animation Children
56097	191773	Up in the Sky (2016)	Adventure Children Fantasy
36666	150010	Bo (2010)	Drama
34060	144005	Sleepaway Camp IV: The Survivor (1992)	Horror
63075	206943	Cássia (2015)	Documentary
54199	187611	Adrift (2018)	Drama
23004	116748	Deadline (2012)	Drama Mystery Thriller
3368	3463	Last Resort (National Lampoon's Last Resort) (...)	Comedy
77385	257669	Honor Among Thieves (2021)	Western

Step 5: Create a Function to Get Movie Recommendations

This step defines the `recommend_movies()` function, which takes a movie title as input and returns the top-N most similar movies using cosine similarity. The function first searches the dataset for matching titles, checks that the movie exists in the similarity matrix, and then retrieves its similarity scores against all other movies. After sorting the scores from highest to lowest, it selects the top recommendations and merges them

with movie details such as title and genres. The result is a clean, easy-to-read DataFrame showing the most similar movies based on user rating patterns.

In [7]:

```
def recommend_movies(movie_title, n_recommendations=5):
    """
    Return the top-N most similar movies to the input movie based on cosine
    similarity.

    Parameters:
        movie_title (str): The movie title (partial or full) to search for.
        n_recommendations (int): Number of recommendations to return.

    Returns:
        DataFrame: A table containing recommended movies, genres, and similarity scores.

    # --- Step 1: Find matching movie titles in the dataset ---
    # Allows partial search (case-insensitive).
    # Example: searching "Toy Stor" will still match "Toy Story (1995)".
    movie_match = movies[movies['title'].str.contains(movie_title, case=False)]

    # If no movie titles match the search input, stop and notify the user.
    if movie_match.empty:
        print(f"Movie '{movie_title}' not found.")
        return None

    # Extract the movieId of the first matched entry.
    # If multiple movies match, this picks the first one.
    movie_id = movie_match.iloc[0]['movieId']
    full_title = movie_match.iloc[0]['title']

    # --- Step 2: Check if similarity data exists for this movie ---
    # Only movies included in the cosine similarity matrix can be recommended.
    if movie_id not in movie_similarity_df.index:
        print(f"Movie '{full_title}' has no similarity data.")
        return None

    # --- Step 3: Retrieve similarity scores for the selected movie ---
    # movie_similarity_df.loc[movie_id] returns similarity between this movie and others.
    # Drop the movie itself (similarity = 1.0) to avoid self-recommendation.
    sims = movie_similarity_df.loc[movie_id].drop(movie_id).sort_values(ascending=False)

    # --- Step 4: Select top-N most similar movies ---
    # This gives the highest cosine similarity values.
    top = sims.head(n_recommendations)

    # --- Step 5: Build a DataFrame containing movie details + similarity scores ---
    recs = (
        movies[movies['movieId'].isin(top.index)] # keep only recommended movies
        .set_index('movieId') # index by movieId for alignment
        .loc[top.index] # ensure the output is sorted by similarity
        .assign(similarity_score=top.values) # add the cosine similarity scores
    )
```



```
# Display a clean message for the user
print(f"Top {n_recommendations} recommendations for '{full_title}':")

# Return a final DataFrame with useful columns
return recs[['title', 'genres', 'similarity_score']]
```

In [8]:

```
# Step 6: Get Top 5 Recommendations for a Chosen Movie

movie_to_recommend_for = "Air Bud" # <- change this to any title you want
top5_recs = recommend_movies(movie_to_recommend_for, n_recommendations=10)

top5_recs
```

Top 10 recommendations for 'Air Bud (1997)':

Out[8]:

	title	genres	similarity_score
movieid			
2152	Air Bud: Golden Receiver (1998)	Children Comedy	0.468589
609	Homeward Bound II: Lost in San Francisco (1996)	Adventure Children	0.411477
1474	Jungle2Jungle (a.k.a. Jungle 2 Jungle) (1997)	Children Comedy	0.398151
1702	Flubber (1997)	Children Comedy Fantasy	0.383830
1015	Homeward Bound: The Incredible Journey (1993)	Adventure Children Drama	0.376908
1021	Angels in the Outfield (1994)	Children Comedy	0.370525
1588	George of the Jungle (1997)	Children Comedy	0.363011
1005	D3: The Mighty Ducks (1996)	Children Comedy	0.362909
2123	All Dogs Go to Heaven (1989)	Animation Children Comedy Drama Fantasy	0.362382
2082	Mighty Ducks, The (1992)	Children Comedy	0.359258

Resources and Works Cited

The development of this movie recommender system was informed by several publicly available resources that demonstrate how to apply cosine similarity, pivot rating matrices, and implement collaborative filtering techniques. The following works provided useful guidance on methodology, structure, and best practices for building recommendation engines:

- M. J. Mohan. *Movie Recommendation System Using Cosine Similarity*. Medium (2023).
<https://medium.com/@jishnumohan481/movie-recommendation-system-using-cosine-similarity-35f8667b6471>
- M. Ayman. *Recommendation System Using Cosine Similarity*. Kaggle Notebook (2022).
<https://www.kaggle.com/code/muhammadayman/recommendation-system-using-cosine-similarity>
- Matec Conferences. *A Study on Movie Recommendation Techniques Using Cosine Similarity* (2024).
https://www.matec-conferences.org/articles/mateconf/pdf/2024/04/mateconf_icmed2024_01071.pdf